

**DESIGN AND DEVELOP A CLOUD-BASED INDUSTRIAL DG
MONITORING AND ANALYTICS SYSTEM. THE SYSTEM
COLLECTS AND ANALYZES DIESEL GENERATOR PERFORMANCE
DATA USING CLOUD TECHNOLOGIES FOR REMOTE
MONITORING APPLICATIONS.**

A PROJECT REPORT

Submitted by

MATHESH NISHANTH B – 24MER030

MOHAMMED SABEER S – 24MER031

PRAAVIN RAJAA G – 24MER039

PRASANNA T – 24MER042

DEVA PRIYAN R – 24MEL064

SATHISH KUMAR K – 24MEL067

DEPARTMENT OF MECHANICAL ENGINEERING



(Autonomous)

PERUNDURAI ERODE – 638 060

1. Project Overview

The Cloud-Based Industrial Diesel Generator (DG) Monitoring and Analytics System is designed to remotely monitor the operating condition and performance of diesel generators using IoT and cloud technologies. The system collects real-time data from sensors installed on the DG, including voltage, current, frequency, engine temperature, fuel level, battery voltage, power output, and operating hours. An IoT controller processes the sensor data and transmits it to a cloud platform through an internet connection. The cloud platform stores, analyzes, and visualizes the data using dashboards and performance graphs. Operators can remotely monitor the DG status, identify abnormal conditions, analyze historical performance, and receive alerts when critical parameters exceed predefined limits. This system helps improve generator reliability, fuel management, preventive maintenance, operational efficiency, and overall industrial power management.

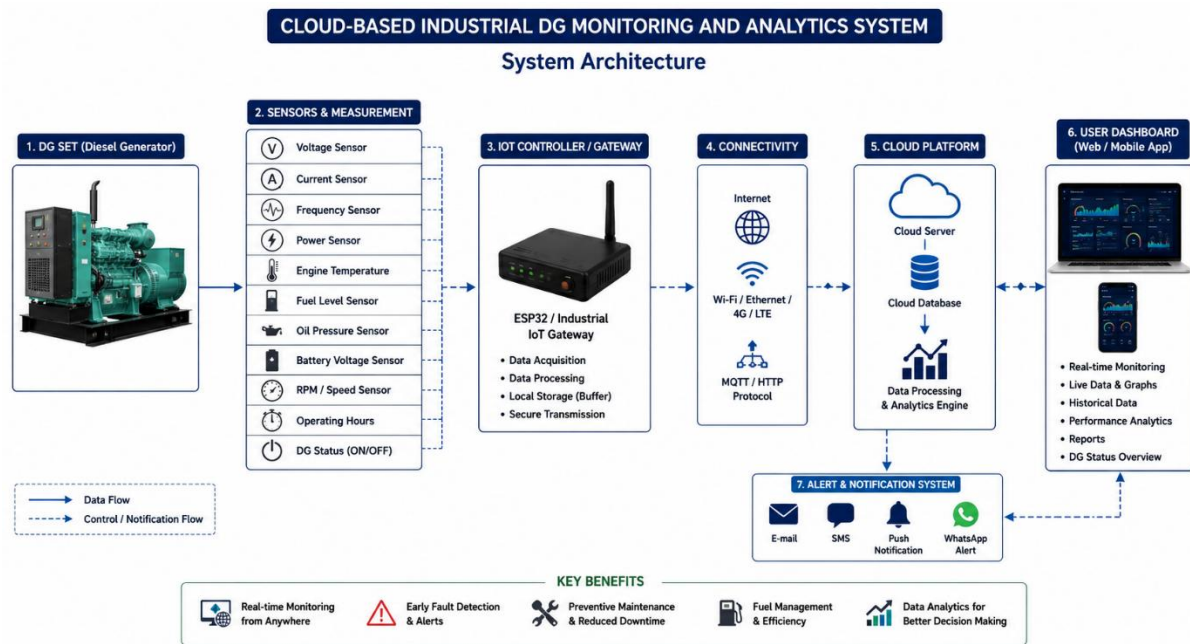
2. Problem Statement

Industrial Diesel Generators (DGs) are widely used as backup and continuous power sources in factories, construction sites, hospitals, commercial buildings, and other critical facilities. Conventional DG monitoring mainly depends on manual inspection, where operators periodically check parameters such as fuel level, voltage, current, temperature, frequency, and operating hours. This approach can result in delayed detection of faults, inefficient fuel usage, unexpected equipment failures, and increased maintenance costs. In addition, the absence of centralized data storage makes it difficult to analyze long-term generator performance and identify operational trends. Therefore, there is a need for a cloud-based DG monitoring and analytics system that continuously collects generator data, transmits it to the cloud, analyzes performance remotely, and provides real-time monitoring and alerts to improve reliability, efficiency, and preventive maintenance.

3. Proposed Solution

The proposed system develops a Cloud-Based Industrial Diesel Generator (DG) Monitoring and Analytics System that continuously monitors the operating condition and performance of a diesel generator using IoT sensors and cloud technology. Sensors are installed to measure important parameters such as voltage, current, frequency, engine temperature, fuel level, battery voltage, power output, and operating hours. An IoT-enabled controller collects and processes the sensor data before transmitting it securely to a cloud platform through an internet connection. The cloud platform stores the data and provides real-time dashboards, graphs, historical records, and performance analytics. If abnormal conditions such as overheating, low fuel, overload, abnormal voltage, or low battery voltage are detected, the system can generate alerts for the operator. This solution enables remote monitoring, early fault detection, predictive maintenance, improved fuel management, reduced downtime, and better overall DG operating efficiency.

4. System Architecture

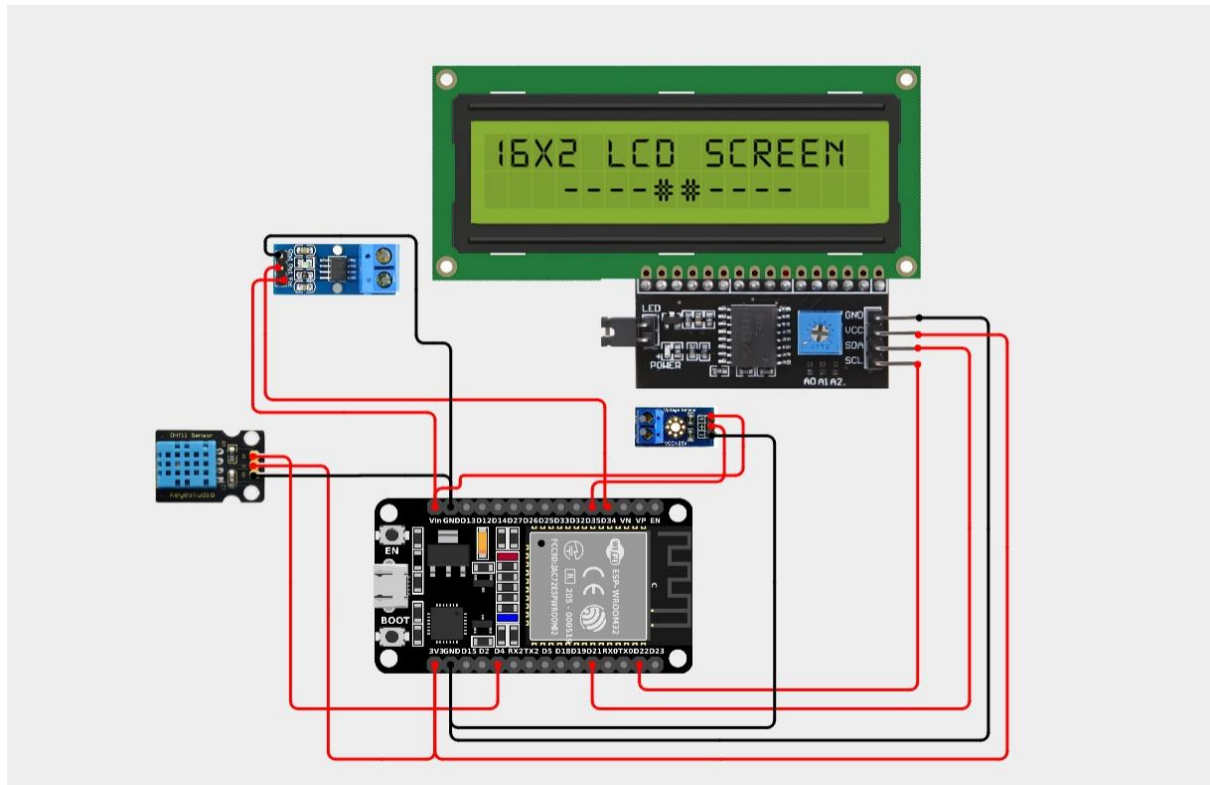


5. Circuit Table

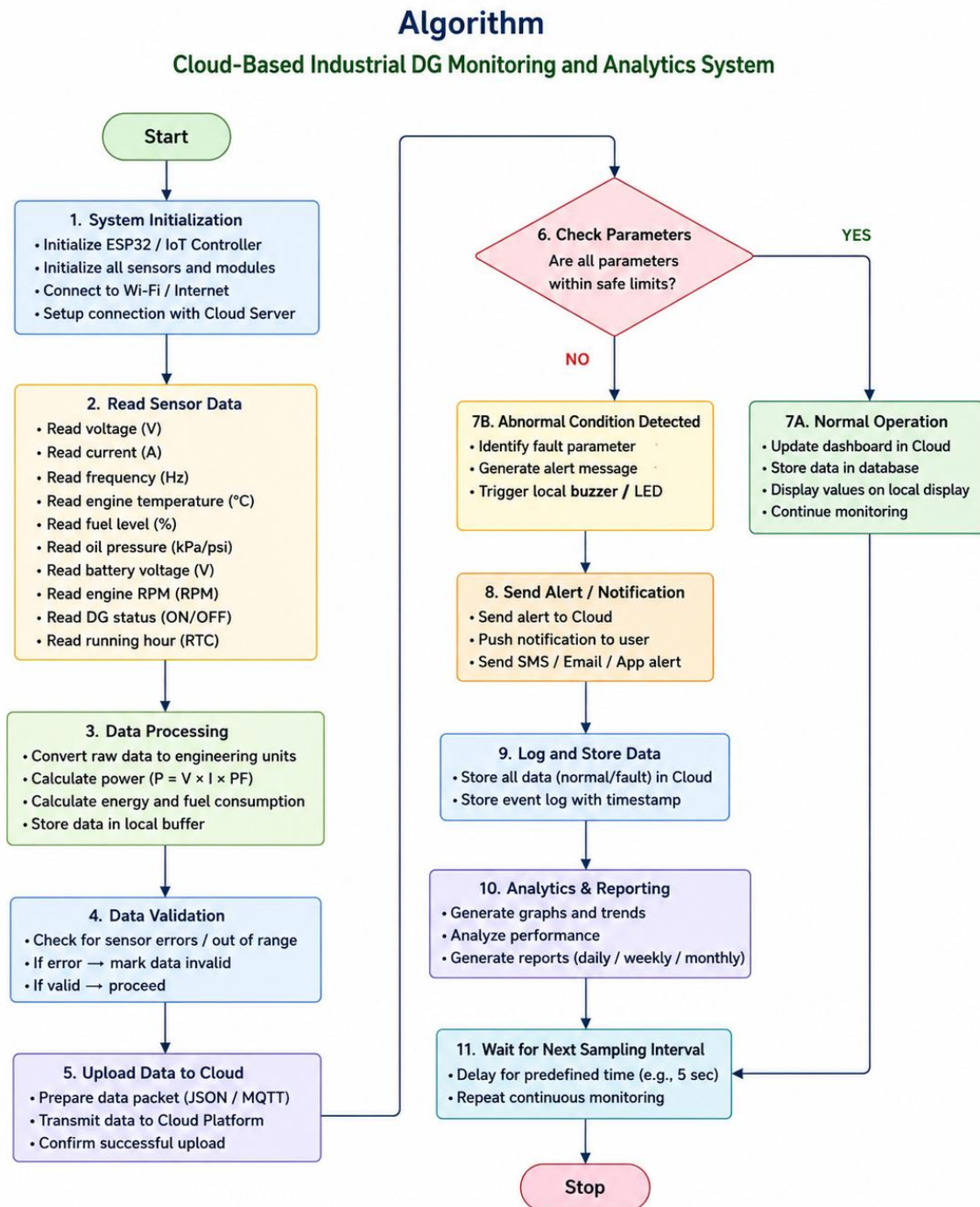
S.No.	Component / Sensor	Parameter Measured	Connection to Controller	Purpose
1	ESP32 / IoT Controller	—	Main controller	Collects, processes, and transmits sensor data
2	Voltage Sensor / ZMPT101B	DG Output Voltage	ADC Pin	Measures generator output voltage
3	Current Sensor / ACS712	Load Current	ADC Pin	Measures current drawn by the load
4	Frequency Sensor	Generator Frequency	Digital Pin	Monitors DG frequency in Hz
5	Temperature Sensor / DS18B20	Engine Temperature	GPIO Pin	Monitors engine temperature

S.No.	Component / Sensor	Parameter Measured	Connection to Controller	Purpose
6	Fuel Level Sensor	Fuel Level	ADC Pin	Measures remaining diesel level
7	Oil Pressure Sensor	Oil Pressure	ADC Pin	Detects low/high engine oil pressure
8	Battery Voltage Sensor	Battery Voltage	ADC Pin	Monitors DG battery condition
9	RPM Sensor	Engine Speed	Digital/Interrupt Pin	Measures engine RPM
10	RTC Module	Operating Time	PC	Records DG operating time
11	LCD/OLED Display	Sensor Data	PC	Displays local DG parameters
12	Wi-Fi / 4G Module	Internet Connectivity	UART / Wi-Fi	Sends data to cloud platform
13	Buzzer / LED	Fault Indication	Digital Pin	Provides local warning during abnormal conditions
14	Cloud Platform	Data Storage & Analytics	Internet	Stores, analyzes, and visualizes DG data
15	Power Supply Module	DC Power	5V / 3.3V	Supplies regulated power to the control circuit

6. Circuit Design



7. Algorithm



8. Coding

```
// =====  
  
// INDUSTRIAL DG MONITORING AND ANALYTICS SYSTEM  
  
// ESP32 + ACS712 + Voltage Sensor + DHT11 + OLED  
  
// + ThingSpeak Cloud  
  
// =====  
  
#include <WiFi.h>  
  
#include <HTTPClient.h>  
  
#include <Wire.h>  
  
#include <Adafruit_GFX.h>  
  
#include <Adafruit_SSD1306.h>  
  
#include <DHT.h>  
  
  
// =====  
  
// WIFI  
  
// =====  
  
const char* WIFI_SSID = "kowsi";  
const char* WIFI_PASSWORD = "kowsi @ 01";  
  
  
// =====  
  
// THINGSPEAK  
  
// =====  
  
const char* THINGSPEAK_API_KEY =  
    "52SYSQ1DR8W0JC8D";
```

```
const char* THINGSPEAK_SERVER =  
    "http://api.thingspeak.com/update";
```

```
// =====  
// OLED  
// =====
```

```
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64
```

```
#define OLED_SDA 21  
#define OLED_SCL 22  
#define OLED_ADDRESS 0x3C
```

```
Adafruit_SSD1306 display(  
    SCREEN_WIDTH,  
    SCREEN_HEIGHT,  
    &Wire,  
    -1  
);
```

```
// =====  
// DHT11  
// =====
```

```
#define DHT_PIN 4  
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```



```
// =====  
// ACS712  
// =====
```

```
#define CURRENT_PIN 34
```

```
// ACS712-20A = 0.100 V/A
```

```
// ACS712-5A = 0.185 V/A
```

```
// ACS712-30A = 0.066 V/A
```

```
#define ACS712_SENSITIVITY 0.100
```

```
// =====  
// VOLTAGE SENSOR  
// =====
```

```
#define VOLTAGE_PIN 35
```

```
#define VOLTAGE_SENSOR_RATIO 5.0
```

```
// =====  
// ADC  
// =====
```

```
#define ADC_MAX 4095.0
```

```
#define ADC_REFERENCE 3.3
```

```
// =====
```

```
// VARIABLES
```

```
// =====
```

```
float voltageValue = 0.0;
```

```
float currentValue = 0.0;
```

```
float temperatureValue = 0.0;
```

```
float humidityValue = 0.0;
```

```
float powerValue = 0.0;
```

```
// =====
```

```
// READ CURRENT
```

```
// =====
```

```
float readCurrent()
```

```
{
```

```
    const int samples = 300;
```

```
    float sum = 0;
```

```
    for (int i = 0; i < samples; i++)
```

```
    {
```

```
        int adcValue = analogRead(CURRENT_PIN);
```

```
        float voltage =
```

```
            (adcValue / ADC_MAX) * ADC_REFERENCE;
```

```
        sum += voltage;
```

```
        delayMicroseconds(500);
```

```
}
```

```
float averageVoltage =
```

```
    sum / samples;
```

```
// Approximate ACS712 zero-current voltage
```

```
float zeroCurrentVoltage = 2.5;
```

```
float current =
```

```
    (averageVoltage - zeroCurrentVoltage)
```

```
    / ACS712_SENSITIVITY;
```

```
if (current < 0)
```

```
{
```

```
    current = -current;
```

```
}
```

```
if (current < 0.05)
```

```
{
```

```
    current = 0;
```

```
}
```

```
return current;
```

```
}
```

```
// =====
```

```
// READ VOLTAGE
```

```
// =====
```

```
float readVoltage()
{
    const int samples = 50;

    long total = 0;

    for (int i = 0; i < samples; i++)
    {
        total += analogRead(VOLTAGE_PIN);

        delay(2);
    }

    float averageADC =
        total / (float)samples;

    float sensorVoltage =
        (averageADC / ADC_MAX)
        * ADC_REFERENCE;

    float inputVoltage =
        sensorVoltage
        * VOLTAGE_SENSOR_RATIO;

    return inputVoltage;
}

// =====
// OLED DISPLAY
```

```
// =====
```

```
void displayData()
{
    display.clearDisplay();

    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);

    display.setCursor(0, 0);
    display.println("INDUSTRIAL DG MONITOR");

    display.setCursor(0, 12);
    display.print("V: ");
    display.print(voltageValue, 1);
    display.println(" V");

    display.setCursor(0, 23);
    display.print("I: ");
    display.print(currentValue, 2);
    display.println(" A");

    display.setCursor(0, 34);
    display.print("P: ");
    display.print(powerValue, 1);
    display.println(" W");

    display.setCursor(0, 45);
    display.print("T:");
```

```
display.print(temperatureValue, 1);  
display.print("C ");
```

```
display.print("H:");  
display.print(humidityValue, 0);  
display.print("%");
```

```
display.setCursor(0, 56);
```

```
if (WiFi.status() == WL_CONNECTED)  
{  
    display.print("WiFi: ONLINE");  
}  
else  
{  
    display.print("WiFi: OFFLINE");  
}
```

```
display.display();  
}
```

```
// =====  
// CONNECT WIFI  
// =====
```

```
void connectWiFi()  
{  
    Serial.print("Connecting to WiFi");
```

```
WiFi.begin(
  WIFI_SSID,
  WIFI_PASSWORD
);

int attempts = 0;

while (
  WiFi.status() != WL_CONNECTED &&
  attempts < 30
)
{
  delay(500);

  Serial.print(".");

  attempts++;
}

Serial.println();

if (WiFi.status() == WL_CONNECTED)
{
  Serial.println("WiFi connected!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}
else
```

```

    {
        Serial.println("WiFi connection failed.");
    }
}

// =====
// SEND DATA TO THINGSPEAK
// =====

void sendToThingSpeak()
{
    if (WiFi.status() != WL_CONNECTED)
    {
        Serial.println(
            "WiFi not connected. Data not sent."
        );

        return;
    }

    HTTPClient http;

    String url =
        String(THINGSPEAK_SERVER) +
        "?api_key=" +
        String(THINGSPEAK_API_KEY) +
        "&field1=" +
        String(voltageValue, 2) +
        "&field2=" +

```



```
String(currentValue, 2) +  
"&field3=" +  
String(temperatureValue, 2) +  
"&field4=" +  
String(humidityValue, 2) +  
"&field5=" +  
String(powerValue, 2);  
  
Serial.println("Sending data to ThingSpeak...");  
  
http.begin(url);  
  
int httpCode = http.GET();  
  
if (httpCode > 0)  
{  
    String response =  
        http.getString();  
  
    Serial.print("ThingSpeak response: ");  
    Serial.println(response);  
  
    if (response != "0")  
    {  
        Serial.println(  
            "Data uploaded successfully!"  
        );  
    }  
    else
```

```
{
  Serial.println(
    "ThingSpeak upload failed."
  );
}
}
else
{
  Serial.print(
    "HTTP Error: "
  );

  Serial.println(httpCode);
}

http.end();
}

// =====
// SETUP
// =====

void setup()
{
  Serial.begin(115200);

  delay(1000);

  // ----- OLED -----
```

```
Wire.begin(
    OLED_SDA,
    OLED_SCL
);

if (!display.begin(
    SSD1306_SWITCHCAPVCC,
    OLED_ADDRESS
))
{
    Serial.println(
        "OLED initialization failed!"
    );

    while (1)
    {
        delay(1000);
    }
}

display.clearDisplay();

display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);

display.setCursor(15, 20);
display.println("INDUSTRIAL DG");
```

```
display.setCursor(20, 35);  
display.println("MONITORING");
```

```
display.display();
```

```
delay(2000);
```

```
// ----- DHT -----
```

```
dht.begin();
```

```
// ----- ADC -----
```

```
analogReadResolution(12);
```

```
analogSetPinAttenuation(  
    CURRENT_PIN,  
    ADC_11db  
);
```

```
analogSetPinAttenuation(  
    VOLTAGE_PIN,  
    ADC_11db  
);
```

```
// ----- WIFI -----
```

```
connectWiFi();
```

```
    delay(1000);
}

// =====

// LOOP

// =====

void loop()
{
    Serial.println();
    Serial.println(
        "Reading DG parameters..."
    );

    // ----- CURRENT -----

    currentValue =
        readCurrent();

    // ----- VOLTAGE -----

    voltageValue =
        readVoltage();

    // ----- DHT11 -----

    float newHumidity =
        dht.readHumidity();
```

```
float newTemperature =
    dht.readTemperature();

if (!isnan(newHumidity) &&
    !isnan(newTemperature))
{
    humidityValue =
        newHumidity;

    temperatureValue =
        newTemperature;
}
else
{
    Serial.println(
        "DHT11 reading failed!"
    );
}

// ----- POWER -----

powerValue =
    voltageValue *
    currentValue;

// ----- SERIAL -----

Serial.println(
    "-----"
```

```
);
```

```
Serial.print("Voltage: ");
```

```
Serial.print(voltageValue, 2);
```

```
Serial.println(" V");
```

```
Serial.print("Current: ");
```

```
Serial.print(currentValue, 2);
```

```
Serial.println(" A");
```

```
Serial.print("Power: ");
```

```
Serial.print(powerValue, 2);
```

```
Serial.println(" W");
```

```
Serial.print("Temperature: ");
```

```
Serial.print(temperatureValue, 1);
```

```
Serial.println(" C");
```

```
Serial.print("Humidity: ");
```

```
Serial.print(humidityValue, 1);
```

```
Serial.println(" %");
```

```
// ----- OLED -----
```

```
displayData();
```

```
// ----- THINGSPEAK -----
```

```
sendToThingSpeak();
```

```
// ThingSpeak requires at least  
// about 15 seconds between updates  
delay(20000);  
}
```


9. System Working

The Cloud-Based Industrial DG Monitoring and Analytics System continuously monitors the performance of the diesel generator using multiple sensors. The sensors measure parameters such as voltage, current, frequency, engine temperature, fuel level, oil pressure, battery voltage, RPM, and operating hours. These sensor signals are collected by an ESP32/IoT controller, where the raw data is processed and converted into suitable engineering values. The controller then transmits the data to the cloud platform through Wi-Fi, Ethernet, or 4G/LTE connectivity using a suitable communication protocol such as MQTT or HTTP.

The cloud platform stores the received data in a database and performs real-time analysis to identify abnormal operating conditions. The monitored parameters are displayed on a web or mobile dashboard using numerical values, graphs, and performance trends. If a parameter exceeds its predefined safe limit, the system generates an alert or notification to inform the operator. Historical data can also be analyzed to study fuel consumption, load patterns, generator efficiency, operating hours, and maintenance requirements. This continuous monitoring enables remote supervision, early fault detection, preventive maintenance, reduced downtime, and improved overall DG performance.

10. Bill of Materials

S. No.	Component	Specification / Example	Quantity	Purpose
1	ESP32 Development Board	ESP32 Wi-Fi/Bluetooth MCU	1	Main controller and cloud communication
2	Voltage Sensor	AC voltage sensor / ZMPT101B	1	Measures DG output voltage
3	Current Sensor	ACS712 / CT sensor	1	Measures generator load current
4	Fuel Level Sensor	Ultrasonic / Float sensor	1	Monitors diesel fuel level
5	Temperature Sensor	DS18B20 / Industrial temperature sensor	1	Monitors engine/coolant temperature
6	Vibration Sensor	SW-420 / Industrial vibration sensor	1	Detects abnormal engine vibration
7	RPM Sensor	Hall-effect / Inductive proximity sensor	1	Measures engine speed

S. No.	Component	Specification / Example	Quantity	Purpose
8	Battery Voltage Sensor	Voltage divider module	1	Monitors DG battery condition
9	Oil Pressure Sensor	0–10 bar pressure sensor	1	Monitors engine oil pressure
10	Power/Energy Meter	Modbus energy meter	1	Measures power, energy and electrical parameters
11	RS485 Module	MAX485	1	Communication with Modbus sensors/meters
12	Wi-Fi/GSM Communication	ESP32 Wi-Fi / 4G GSM module	1	Transfers data to cloud
13	Cloud Platform	ThingSpeak / MQTT Cloud / AWS IoT	1	Data storage, visualization and analytics
14	LCD/OLED Display	16×2 LCD / 0.96-inch OLED	1	Local display of DG parameters
15	Buzzer	5 V active buzzer	1	Provides fault/alarm indication
16	Relay Module	5 V, 2/4 channel	1	Alarm/control switching
17	Power Supply	12 V DC / 5 V DC regulated supply	1	Powers monitoring circuit
18	DC-DC Buck Converter	LM2596	1	Converts 12 V to required DC voltage
19	Fuse & Protection	Fuse, MCB, TVS/protection components	1 set	Circuit protection
20	Connecting Wires	Jumper/industrial wires	1 set	Electrical connections
21	Terminal Blocks	Screw terminal connectors	1 set	Secure sensor connections
22	PCB / Prototype Board	Suitable size	1	Circuit assembly

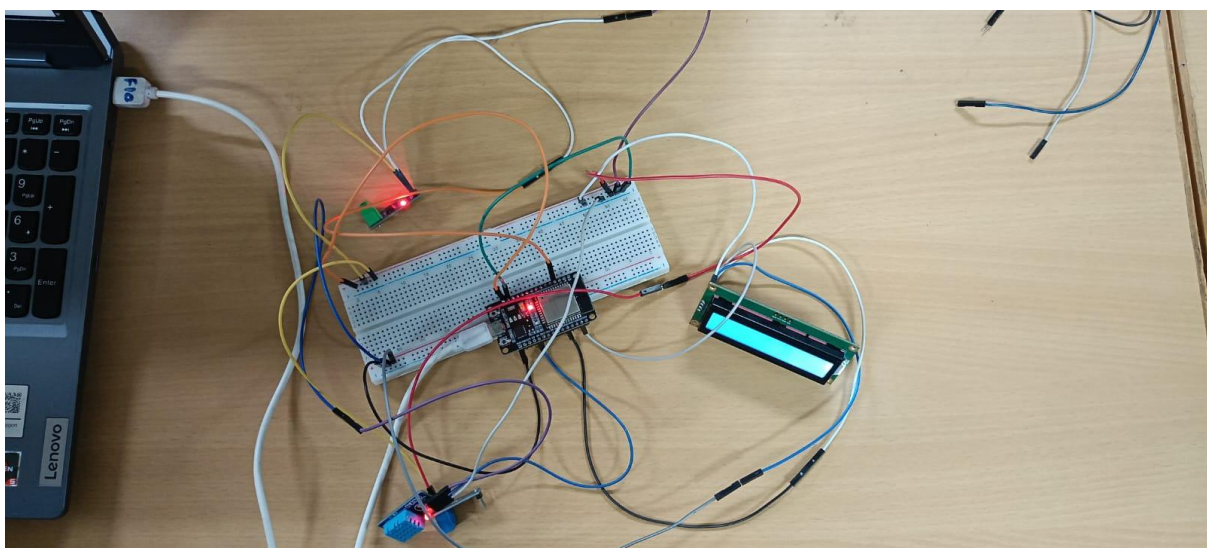
11. Prototype Implementation

The prototype of the Cloud-Based Industrial DG Monitoring and Analytics System is developed by integrating multiple sensors with an ESP32 microcontroller and a cloud communication platform. The sensors are connected to the diesel generator to continuously measure important operating parameters such as output voltage, load current, engine temperature, fuel level, RPM, battery voltage, vibration, and oil pressure. A Modbus energy meter can be connected through an RS485 interface to obtain electrical parameters such as power and energy consumption.

The ESP32 acts as the central processing unit of the prototype. It collects sensor readings, performs basic data processing, and transmits the measured data to the cloud through Wi-Fi or a suitable GSM/4G communication module. The cloud platform stores the received data and displays it through real-time dashboards and graphical representations. Threshold values are configured for critical parameters, allowing the system to generate alerts when abnormal operating conditions are detected.

For local monitoring, an LCD/OLED display can be used to show important DG parameters, while a buzzer provides an immediate indication of fault conditions. The complete electronic circuit is installed inside a protective enclosure, with appropriate power regulation, fuses, terminal blocks, and electrical isolation. The prototype is tested under different generator operating conditions to verify sensor accuracy, data transmission, cloud visualization, and alarm functionality.

Overall, the implemented prototype demonstrates real-time remote monitoring, cloud-based data logging, performance analysis, and early fault detection of an industrial diesel generator.



12. Results & Performance Analysis

The developed Cloud-Based Industrial DG Monitoring and Analytics System was successfully implemented and tested under different operating conditions of the diesel generator. The system continuously monitored important parameters such as voltage, current, power, fuel level, engine temperature, RPM, battery voltage, vibration, and oil pressure. The ESP32 controller collected the sensor data and transmitted it to the cloud platform through an internet connection. The received data was stored and displayed on a cloud dashboard in the form of real-time values and graphical trends, allowing users to monitor the generator remotely.

During testing, the system effectively detected variations in generator load and operating conditions. Changes in voltage and current were continuously monitored, while fuel-level sensing provided information about diesel availability and consumption. Engine temperature, RPM, vibration, and oil pressure were also monitored to identify abnormal operating conditions. When the measured value exceeded the predefined threshold, the system generated an alarm indication to alert the operator. The cloud-based monitoring feature reduced the need for continuous physical inspection and provided easy access to generator performance data from a remote location.

Overall, the prototype demonstrated reliable performance in real-time data acquisition, cloud data transmission, remote monitoring, parameter visualization, and fault indication. The system improves generator maintenance and operational efficiency by providing continuous performance information and early warnings of possible problems. The proposed system can be further enhanced by incorporating predictive maintenance algorithms, machine learning, automated alerts, fuel consumption analysis, and industrial-grade sensors for large-scale industrial applications.

